



Neural Network (perceptron)

Section 4

Created by: Enas Rajaie

Review

How to train Perceptron?

- **Step1:** Setting initial values for weights.
- **Step2:** calculate the inputs value by multiplying the input by its corresponding weight (like: $x_1 * w_1$)
- **Step3:** Sum all the values resulting from multiplying the inputs by their weights
 $b * w_b + x_1 * w_1 + x_2 * w_2 + \dots + x_n * w_n$
- **Step 4:** Pass the resulting value to the appropriate activation function.



Review

How to train Perceptron?

- **Step 5:** Compare the resulting value with the actual value. If they match, input the following data for training (repeat from the step two). If they do not match, adjust the weight values using this formula:

$$\text{error} = y_{\text{true}} - y_{\text{pre}}$$

$$w_{\text{new}} = w_{\text{old}} - (\eta * \text{error} * X) \quad \text{x: input corresponds to weight.}$$

- **Step 6:** Repeat Steps 2 → 6 for all training samples (epochs) until:
No error occur or maximum iteration reached(If the number of iterations is specified).



Example

Train a perceptron on OR Gate:-

- bias=1, learning rate (η) = 0.1
- $w_1 = 0.0$, $w_2 = 0.0$, $w_{\text{bias}} = 0.0$
- and use step function with threshold= 0
 - If $z \geq 0$ then $y_{\text{pred}} = 1$
 - Else 0

Error = $y_{\text{true}} - y_{\text{pred}}$

$w_{\text{new}} = w_{\text{old}} + (\eta \times \text{error} \times x)$

OR Gate

Epoch	x_1	x_2	b	w_1	w_2	w_b	z	y_{pred}	y_{true}
1	0	0	1	0.0	0.0	0.0	0	1	0
	0	1	1	0.0	0.0	-0.1	-0.1	0	1
	1	0	1	0.0	0.1	0.0	0	1	1
	1	1	1	0.0	0.1	0.0	0.1	1	1
2	0	0	1	0.0	0.1	0.0	0	1	0
	0	1	1	0.0	0.1	-0.1	0	1	1
	1	0	1	0.0	0.1	-0.1	-0.1	0	1
	1	1	1	0.1	0.1	0.0	0.2	1	1
3	0	0	1	0.1	0.1	0.0	0	1	0
	0	1	1	0.1	0.1	-0.1	0	1	1
	1	0	1	0.1	0.1	-0.1	0	1	1
	1	1	1	0.1	0.1	-0.1	0.1	1	1
4	0	0	1	0.1	0.1	-0.1	-0.1	0	0
	0	1	1	0.1	0.1	-0.1	0	1	1
	1	0	1	0.1	0.1	-0.1	0	1	1
	1	1	1	0.1	0.1	-0.1	0.1	1	1

OR Gate Coding

```
File Edit View Tools VCS Window Help numpy.py - main.py
main.py x
Visual layout of bidirectional text can depend on base direction (set in View menu) Choose direction Hide notification Don't show again
1 w1, w2, w_b = 0.0, 0.0, 0.0
2 lr = 0.1
3
4 bias=1
5 x1 = [0, 0, 1, 1]
6 x2 = [0, 1, 0, 1]
7 target = [0, 1, 1, 1]
8 prediction = [0, 0, 0, 0]
9
10 for epoch in range(10):
11     for i in range(4):
12         z = x1[i]*w1 + x2[i]*w2 + bias*w_b
13
14         #pred = 1 if z >= 0 else 0
15
16         if z >= 0:
17             pred = 1
18         else:
19             pred = 0
20
21         prediction[i] = pred
22         error = target[i] - pred
23         if error != 0:
24             w1 = w1 + (lr * error * x1[i])
25             w2 = w2 + (lr * error * x2[i])
26             w_b = w_b + (lr * error * bias)
27
28     print("النتيجة النهائية:")
29     for i in range(4):
30         print(x1[i], x2[i], "->", prediction[i])
31
```

PyCharm 2026.1 available
Update...

OR Gate Coding

1. We initialize the weights w_1 , w_2 , and w_b to 0.0, set the learning rate $lr = 0.1$, and fix the bias input as a constant 1.
2. The input features x_1 and x_2 represent the two binary inputs for the OR gate, while target holds the expected outputs [0, 1, 1, 1].
3. The prediction list is initially set to all zeros, meaning the model has not made any correct predictions yet and will start learning from the first training example.

```
w1, w2, w_b = 0.0, 0.0, 0.0  
lr = 0.1
```

```
bias=1  
x1 = [0, 0, 1, 1]  
x2 = [0, 1, 0, 1]  
target = [0, 1, 1, 1]  
prediction = [0, 0, 0, 0]
```

OR Gate Coding

- 1. The outer loop runs for **10 epochs**, and the inner loop iterates over the **4 training examples** of the OR gate, where for each example it calculates the **weighted sum** $z = x1[i]*w1 + x2[i]*w2 + bias*w_b$.
- 2. The **step activation function** checks if $z \geq 0$ to set $pred = 1$ (otherwise $pred = 0$), stores it in $prediction[i]$, then computes the $error = target[i] - pred$.
- 3. If the error is **not zero**, the weights are updated using the **perceptron learning rule**:
- $w1 = w1 + (lr * error * x1[i])$,
 $w2 = w2 + (lr * error * x2[i])$, and
 $w_b = w_b + (lr * error * bias)$

```
for epoch in range(10):
    for i in range(4):
        z= x1[i]*w1 + x2[i]*w2 + bias*w_b

        if z>=0:                                #pred = 1 if z >= 0 else 0
            pred=1
        else:
            pred=0

        prediction[i]=pred
        error = target[i] - pred
        if error != 0:
            w1= w1 + (lr * error * x1[i])
            w2= w2 + (lr * error * x2[i])
            w_b= w_b + (lr * error * bias)
```


OR Gate Coding

1. The `print("final result " )` displays the header **"Final Result"** before showing the outputs.

2. The for loop runs **4 times** (once for each training example) and prints the **two inputs** `x1[i]` and `x2[i]` followed by the **final prediction** stored in `prediction[i]`.

3. This shows whether the perceptron **successfully learned** the OR gate by comparing the predicted outputs with the target values `[0, 1, 1, 1]`.

```
Print("final result")  
for i in range(4):  
    print(x1[i], x2[i], "->", prediction[i])
```

Make changes to the code

- 1- Changing the Logic Gate
- 2- Change the number of Epochs
- 3. Changing the learning rate
- 4. Changing the Initial Weights
- 5. Change the activation function threshold
- 6- change bias
- 7. Add a print to the learning process (to track updates)
 - `print(f" {x1[i]}, {x2[i]} -> pred={pred} target={target[i]} error={error}")`

Stages of Training a Perceptron

8. Accuracy Calculation

```
# after training
correct = 0
for i in range(4):
    z = x1[i]*w1 + x2[i]*w2 + bias*w_b
    pred = 1 if z >= 0 else 0
    if pred == target[i]:
        correct += 1

accuracy = (correct / 4) * 100
print("accuracy :", accuracy)
```

Task

**Make changes to this code to build a perceptron training
on 3-input OR Gate**





Thank You

